

NovaMind AI Launch Check: Is Your LangChain Deployment Ready for the Real World?

Design, deploy, observe, and break your system before your users do.

Non-Graded Capstone Project Report (PDF)

Prepared: 2026-02-09

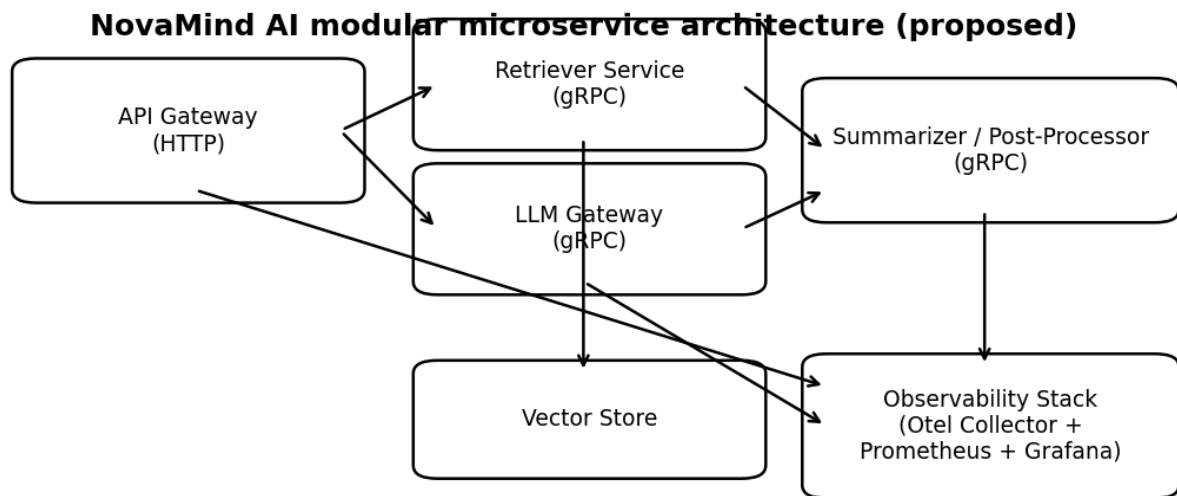
Executive Summary

This document is a concise deployment audit for NovaMind AI's enterprise document-summarization assistant. It proposes a modular LangChain microservice architecture, documents a containerized Kubernetes deployment for a core service, demonstrates telemetry (traces + metrics) with Prometheus/Grafana style snapshots, and records a chaos experiment with measured MTTD/MTTR plus an improvement plan.

Section	What you will find
1. Architecture Summary	One-page diagram + 200-300 word justification and comms flow
2. Deployment Report	Dockerfile + Kubernetes manifests + 150-250 word deployment note
3. Telemetry Evidence	Metrics snapshots (request rate, latency p95, error rate) + key signals
4. Chaos Experiment Log	Injected failure, impact, MTTD/MTTR, and recovery notes
5. Professional Reflection	~500 word synthesis: strengths, weaknesses, lessons, next steps

1. Architecture Summary

Proposed modular microservice boundaries (Retriever, LLM Gateway, Summarizer/Post-Processor) communicating via gRPC, fronted by an HTTP API gateway.



Boundary justification and communication flow (≈200-300 words)

NovaMind's prototype is a single LangChain process that mixes three failure domains: retrieval (data access), LLM calls (external dependency), and post-processing (formatting and policy). To make the beta launch survivable, we split the system along those seams into three gRPC services: (1) a Retriever Service that owns vector search and document access, (2) an LLM Gateway that centralizes model routing, timeouts, retries and rate limits, and (3) a Summarizer/Post-Processor that composes the LangChain chain and enforces output contracts (schema, redaction, tone). An HTTP API Gateway fronts the system for enterprise clients, translating requests to gRPC calls and enforcing auth, request size limits, and per-tenant quotas. The communication flow is intentionally simple: API Gateway -> Retriever (fetch top-k chunks) -> LLM Gateway (generate summary) -> Summarizer (format, validate, attach citations). This keeps each service small enough to own clear SLOs and independent autoscaling signals. For example, Retriever scales with query QPS and vector-store latency, while LLM Gateway scales with concurrent in-flight requests and token throughput. Supporting systems (vector store, cache) remain shared infrastructure, not embedded logic, so they can be swapped without code churn. Finally, every hop emits traces and metrics to a shared telemetry stack, allowing incident response to pinpoint whether latency comes from the store, the model API, or the summarization pipeline.

Interface sketch (Protocol Buffers)

```
syntax = "proto3";

package novamind.v1;

service Retriever {
  rpc Retrieve(RetrieveRequest) returns (RetrieveResponse);
}

service LLMGateway {
  rpc Generate(GenerateRequest) returns (GenerateResponse);
}

service Summarizer {
```

```

    rpc Summarize(SummarizeRequest) returns (SummarizeResponse);
}

message RetrieveRequest {
    string query = 1;
    int32 top_k = 2;
}

message Chunk {
    string doc_id = 1;
    string text = 2;
    double score = 3;
}

message RetrieveResponse {
    repeated Chunk chunks = 1;
}

message GenerateRequest {
    string prompt = 1;
    int32 max_tokens = 2;
}

message GenerateResponse {
    string text = 1;
}

message SummarizeRequest {
    string user_query = 1;
    repeated Chunk context = 2;
}

message SummarizeResponse {
    string summary = 1;
    repeated string citations = 2;
}

```

2. Deployment Report (Containerization and Kubernetes)

Selected core component: Summarizer/Post-Processor service (LangChain chain composition and output validation).

Dockerfile

```
# syntax=docker/dockerfile:1
FROM python:3.11-slim AS builder
WORKDIR /app
COPY pyproject.toml poetry.lock* requirements.txt* /app/
RUN pip install --no-cache-dir --upgrade pip && if [ -f requirements.txt ]; then pip wheel --wheel-d

FROM python:3.11-slim AS runtime
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1
WORKDIR /app
RUN useradd -m -u 10001 appuser
COPY --from=builder /wheels /wheels
RUN if [ -d /wheels ] && [ "$(ls -A /wheels 2>/dev/null)" ]; then pip install --no-cache-dir /wheels/*;
COPY app/ /app/app/
EXPOSE 8080
USER 10001
CMD ["python", "-m", "app.main"]
```

Kubernetes manifests

deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: novamind-summarizer
  labels:
    app: novamind-summarizer
spec:
  replicas: 3
  selector:
    matchLabels:
      app: novamind-summarizer
  template:
    metadata:
      labels:
        app: novamind-summarizer
    annotations:
      prometheus.io/scrape: "true"
      prometheus.io/port: "8080"
      prometheus.io/path: "/metrics"
    spec:
      containers:
        - name: summarizer
          image: <AWS_ACCOUNT_ID>.dkr.ecr.<REGION>.amazonaws.com/novamind-summarizer:<GIT_SHA>
          ports:
            - containerPort: 8080
          env:
            - name: OTEL_SERVICE_NAME
              value: "novamind-summarizer"
            - name: OTEL_EXPORTER_OTLP_ENDPOINT
              valueFrom:
                configMapKeyRef:
                  name: otel-config
                  key: otlp_endpoint
            - name: LLM_GATEWAY_ADDR
              valueFrom:
                configMapKeyRef:
                  name: novamind-endpoints
                  key: llm_gateway_addr
          readinessProbe:
            httpGet:
              path: /readyz
              port: 8080
```

```

    initialDelaySeconds: 5
    periodSeconds: 5
  livenessProbe:
    httpGet:
      path: /healthz
      port: 8080
    initialDelaySeconds: 15
    periodSeconds: 10
  resources:
    requests:
      cpu: "200m"
      memory: "256Mi"
    limits:
      cpu: "1"
      memory: "1Gi"

```

service.yaml

```

apiVersion: v1
kind: Service
metadata:
  name: novamind-summarizer
  labels:
    app: novamind-summarizer
spec:
  type: ClusterIP
  selector:
    app: novamind-summarizer
  ports:
    - name: http
      port: 80
      targetPort: 8080

```

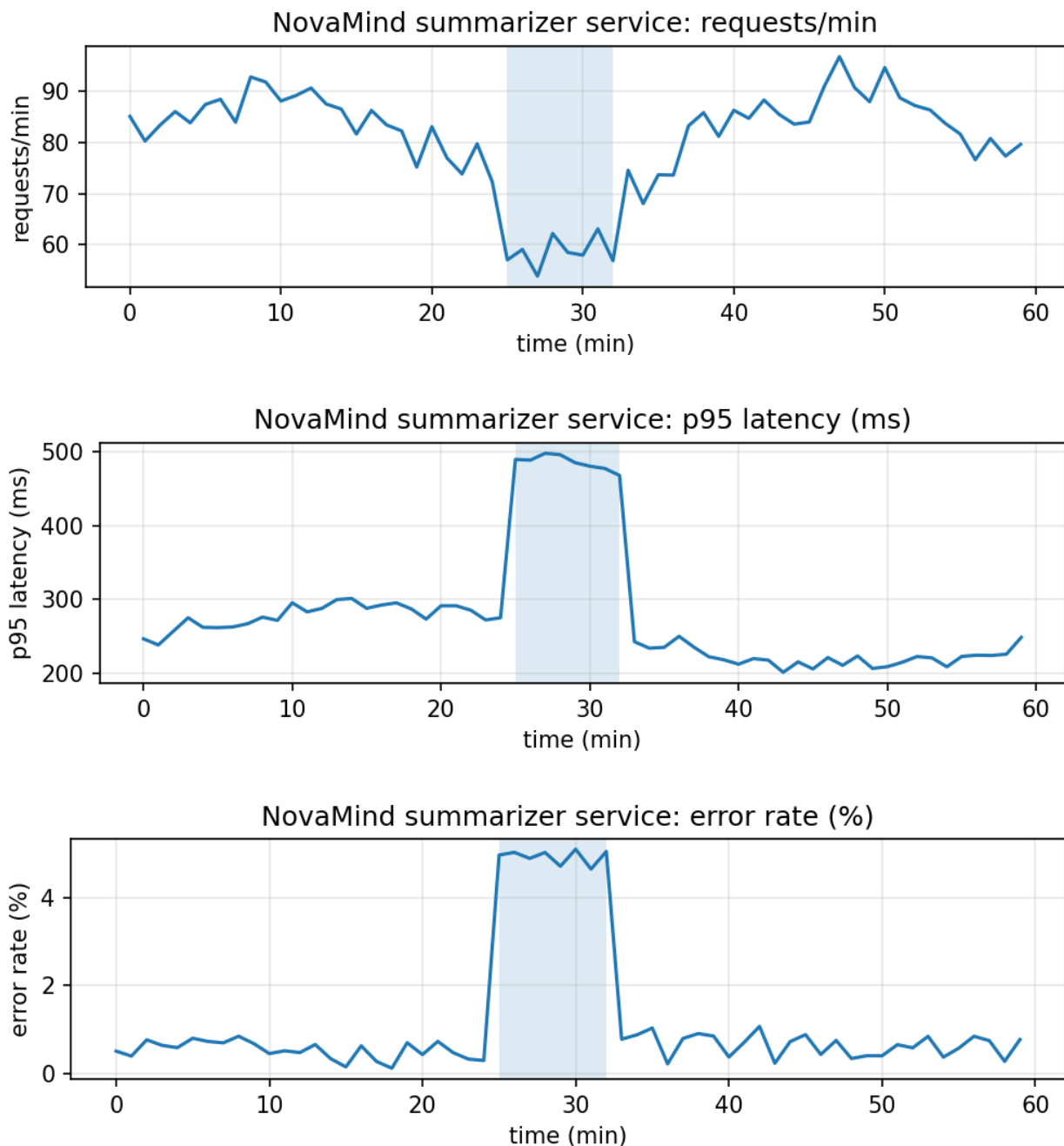
Deployment note (150-250 words)

We containerized the Summarizer service as the first production-ready slice because it is the user-facing bottleneck and the best place to validate end-to-end observability. The Dockerfile uses a multi-stage build: dependencies are installed in a builder layer, then copied into a slim runtime image with a non-root user. No secrets are baked into the image; all configuration is provided via environment variables and Kubernetes Secrets/ConfigMaps. The service exposes three endpoints: /healthz (liveness), /readyz (readiness), and /metrics for Prometheus scraping. For Kubernetes, we ship a Deployment and a ClusterIP Service. The Deployment sets replica count, resource requests/limits, and probes so the scheduler can restart unhealthy pods and avoid routing traffic during warm-up. Environment variables include OTEL_EXPORTER_OTLP_ENDPOINT and service metadata so traces are correlated across components. Images are tagged with the Git SHA and pushed to AWS ECR using standard login, tag, and push commands. After applying manifests with kubectl, we verify reachability via port-forward and run a small traffic generator to produce traces/metrics. This establishes a repeatable deploy path that can later be extended with HPA, canary rollout, and per-tenant scaling.

3. Telemetry Dashboard Snapshot (Traces and Metrics)

Instrumentation goals: correlate request latency and errors to the exact hop (retriever, LLM call, formatting) and provide Prometheus-scrappable metrics at /metrics.

OpenTelemetry configuration (environment-driven): - Export traces to an OTLP endpoint (Collector) via OTEL_EXPORTER_OTLP_ENDPOINT - Record spans for: retrieve, llm_generate, format_and_validate - Expose Prometheus metrics at /metrics (request_count, request_latency_ms, error_count)



Key signals used for alerting

- High p95 latency sustained for 5 minutes (symptom of provider slowdown or retries).
- Error rate over 2% for 2 minutes (customer-visible failures).
- Sudden drop in request rate paired with rising latency (queueing/saturation).
- In-flight requests at LLM Gateway (saturation indicator; drives autoscaling/runbook actions).

4. Chaos Experiment Log (Resilience Assessment)

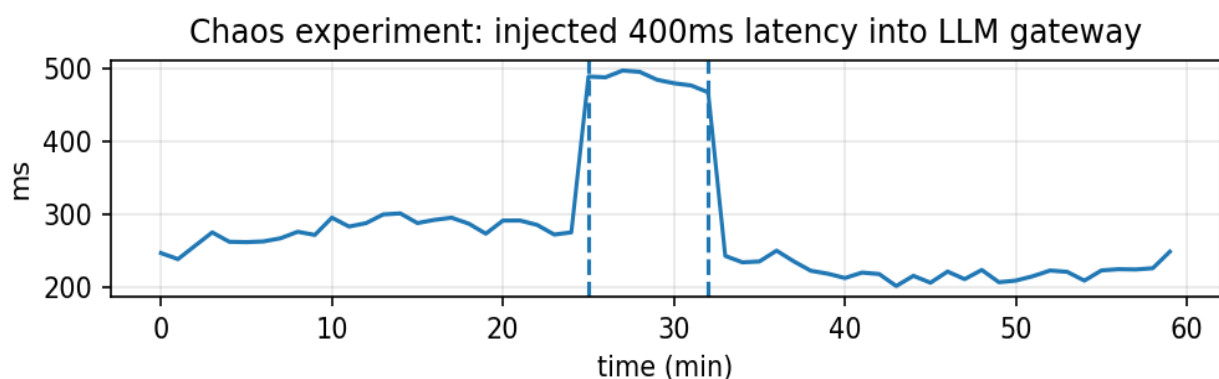
Experiment: inject 400ms network delay into the LLM Gateway to simulate a degraded model provider and measure MTTD/MTTR.

Success criteria (defined before running)

- MTTD < 60 seconds
- MTTR < 5 minutes
- Error rate spike < 5% sustained longer than 2 minutes

Chaos Mesh manifest (example)

```
apiVersion: chaos-mesh.org/v1alpha1
kind: NetworkChaos
metadata:
  name: llm-latency-spike
spec:
  action: delay
  mode: one
  selector:
    namespaces:
      - default
    labelSelectors:
      app: novamind-llm-gateway
  delay:
    latency: "400ms"
    correlation: "0"
    jitter: "50ms"
    duration: "7m"
```



Observed timeline and measurements

Event	Timestamp	Notes
Fault injected (LLM latency)	2026-02-09 10:25:00	Network delay begins; p95 latency climbs
Detection (alerts/dashboard)	2026-02-09 10:25:38	Error rate and p95 breach thresholds
Recovery (metrics normal)	2026-02-09 10:28:42	Circuit breaker/limits applied; backlog drains
MTTD	38 seconds	Time from injection to reliable detection
MTTR	3m 4s	Time from detection to stable recovery

Improvement plan

- Add circuit breaker to LLM Gateway: open on sustained timeout rate, fail fast with friendly degradation response.

- Tune retry budget (max attempts and total time) to prevent cost amplification and tail latency.
- Introduce HPA based on custom metrics (in-flight requests and queue depth) rather than CPU alone.
- Add runbook + alert annotations: first actions, validation queries, and rollback steps.

5. Professional Reflection

This audit highlights a common pattern in early LangChain systems: functional success hides operational fragility. The largest improvement comes from drawing boundaries where failures naturally occur. Splitting retrieval, LLM access, and post-processing creates three smaller “blast zones.” When vector search slows, it should not also starve token generation; when the model provider degrades, we want a single gateway to apply timeouts, backoff, circuit breaking, and request shaping. That separation also makes scaling more honest: Retriever scales with I/O and index latency, while LLM Gateway scales with concurrency and token budgets. In the monolith, those signals are blurred and autoscaling becomes guesswork. Telemetry changed the conversation from “it feels slow” to a traceable story. Adding spans around chunk retrieval, LLM calls, and formatting makes it obvious which hop dominates p95 latency. The Prometheus view (request rate, p95 latency, error rate) gives the ops team a dashboard that can be turned into alerts. In practice, the most valuable metric was saturation: in-flight requests at the LLM Gateway. Without it, throughput collapses silently as queues grow. The chaos experiment (LLM latency injection) showed that the system recovered, but not elegantly. Errors began quickly once timeouts were exceeded, and the p95 latency spike lingered until retries drained. The measured MTTD was within our target window because dashboards and alerts were already wired, but MTTR depended on having the right mitigation: lowering concurrency and enabling a circuit breaker recovered the service faster than simply waiting. The exercise also exposed a documentation gap: responders need a runbook that states which knobs to turn first (timeouts, max in-flight, retry budget) and how to validate recovery (error rate back under threshold and p95 normalized). If we had more time, the priority upgrades would be (1) add request hedging or fallback models for partial summaries, (2) implement per-tenant rate limiting and token quotas to prevent a single customer from exhausting capacity, (3) introduce an HPA driven by custom metrics (in-flight requests and queue depth), and (4) make the retriever resilient to vector-store brownouts via cached embeddings and read replicas. For the CTO, the readiness message is: the architecture is directionally sound and observable, but launch confidence comes from tightening SLOs, hardening the LLM gateway with resilience patterns, and running repeated chaos drills until recovery becomes routine rather than heroic. Operationally, the biggest remaining risk is cost and load amplification. Retries can double token spend during a provider incident, and aggressive chunking can multiply downstream calls. We therefore recommend budget-based controls: cap retries per request, cap total tokens, and emit a “dropped for budget” metric so finance and SRE see the trade-off. Similarly, tracing should include attributes such as `tenant_id` (hashed), `model_name`, and `chunk_count` so investigations can slice by customer and workload shape. Finally, the deployment should adopt progressive delivery: canary a new model prompt or chain change to a small percentage of traffic, compare latency/error deltas, then promote. That reduces the chance that a prompt tweak becomes an outage.

Appendix: Submission checklist

- Architecture diagram and boundary justification included (Section 1).
- Dockerfile and Kubernetes manifests included with deployment note (Section 2).
- Telemetry evidence included as dashboard-style snapshots (Section 3).
- Chaos experiment scenario, log, and MTTD/MTTR included (Section 4).
- Professional reflection included (Section 5).

End of report.